



Correction du TP - Révision Base de Données

Partie 1 : Gestion des utilisateurs et privilèges

1. Création de l'utilisateur admin1

```
CREATE USER admin1 IDENTIFIED BY 12345;
```

2. Attribution des droits à admin1

```
GRANT CREATE TABLE TO admin1;  
GRANT UPDATE (nom, prenom) ON Client TO admin1;  
GRANT SELECT ON Hotel TO admin1;
```

3. Vérification des droits accordés

```
SELECT * FROM USER_TAB_PRIVS WHERE GRANTEE = 'ADMIN1';
```

4. Suppression des droits de admin1

```
REVOKE CREATE TABLE, UPDATE ON Client, SELECT ON Hotel FROM admin1;
```

5. Affichage de l'utilisateur connecté et des utilisateurs existants

```
SELECT USER FROM DUAL;  
SELECT * FROM DBA_USERS;
```

6. Suppression de admin1

```
DROP USER admin1 CASCADE;
```

7. Création de user1 avec obligation de changer le mot de passe

```
CREATE USER user1 IDENTIFIED BY user1 PASSWORD EXPIRE;
```

8. Attribution de droit de création de table à user1

```
GRANT CREATE TABLE TO user1;
```

9. Attribution des droits dans tous les schémas

```
GRANT CREATE ANY TABLE TO user1;
```

10. Affichage des tables de user1

```
SELECT TABLE_NAME FROM USER_TABLES;
```

11. Création d'un tablespace tab_hotel

```
CREATE TABLESPACE tab_hotel DATAFILE 'tab_hotel.dbf' SIZE 4M;
```

12. Affectation du tablespace à user1



```
ALTER USER user1 DEFAULT TABLESPACE tab_hotel QUOTA 1M ON tab_hotel;
```

13. Droit de création de tables pour tous les utilisateurs

```
GRANT CREATE TABLE TO PUBLIC;
```

14. Création du profil profig1

```
CREATE PROFILE profig1 LIMIT  
  FAILED_LOGIN_ATTEMPTS 2  
  PASSWORD_LIFE_TIME 60  
  SESSIONS_PER_USER 2  
  IDLE_TIME 2;
```

15. Affectation du profil à user1

```
ALTER USER user1 PROFILE profig1;
```

16. Vérification du profil de user1

```
SELECT * FROM DBA_USERS WHERE USERNAME = 'USER1';
```

17. Vérification des limites après affectation du profil

```
SELECT * FROM DBA_PROFILES WHERE PROFILE = 'PROFIG1';
```

18. Suppression du profil profig1

```
DROP PROFILE profig1;
```

19. Vérification des nouvelles limites après suppression

```
SELECT * FROM DBA_PROFILES WHERE PROFILE = 'DEFAULT';
```

20. Création de la table Clients par user1

```
CREATE TABLE Clients (idClient INT PRIMARY KEY, nom VARCHAR(50), prenom  
  VARCHAR(50));
```

21. Création de l'utilisateur Hot

```
CREATE USER Hot IDENTIFIED BY 12345;
```

22. Droit de sélection sur Clients pour Hot

```
GRANT SELECT ON Clients TO Hot;
```

23. Affichage des tables dynamiques du dictionnaire de données

```
SELECT * FROM DICTIONARY;
```



Partie 2 : Exercice PL/SQL

1. Insertion d'un client en PL/SQL

```
BEGIN
    INSERT INTO Client (idClient, nom, prenom) VALUES (1, 'Dupont', 'Jean');
    COMMIT;
END;
```

2. Procédure pour insérer des chambres

```
CREATE PROCEDURE Insert_Chambres AS
BEGIN
    FOR i IN 1..30 LOOP
        INSERT INTO Chambre (idChambre, type, tarif, idHotel, libre)
        VALUES (i, 'Standard', 100, 1, 'Oui');
    END LOOP;
    COMMIT;
END;
```

3. Calcul de la somme des réservations de l'année en cours

```
DECLARE
    total NUMBER;
BEGIN
    SELECT SUM(montant) INTO total FROM Reservation
    WHERE EXTRACT(YEAR FROM date_arrivee) = EXTRACT(YEAR FROM SYSDATE);
    DBMS_OUTPUT.PUT_LINE('Total des réservations : ' || total);
END;
```

4. Affichage du reste de la division de 2390 par 13 sans utiliser MOD

```
DECLARE
    dividend NUMBER := 2390;
    divisor NUMBER := 13;
    quotient NUMBER;
    remainder NUMBER;
BEGIN
    quotient := TRUNC(dividend / divisor);
    remainder := dividend - (quotient * divisor);
    DBMS_OUTPUT.PUT_LINE('Le reste de la division est : ' || remainder);
END;
```

Partie 3 : Triggers

1. Empêcher la mise à jour des réservations le week-end

```
CREATE TRIGGER Prevent_Weekend_Update
BEFORE UPDATE ON Reservation
FOR EACH ROW
WHEN (TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN'))
BEGIN
    RAISE_APPLICATION_ERROR(-20001, 'Mise à jour interdite le week-end');
END;
```



2. Mise à jour du compteur des réservations

```
CREATE TRIGGER Update_Reservation_Count
AFTER INSERT ON Reservation
FOR EACH ROW
BEGIN
    UPDATE CompteurReservations SET nombre_reservations =
nombre_reservations + 1
    WHERE idHotel = :NEW.idHotel;
END;
```

3. Trigger de sauvegarde des clients

```
CREATE TRIGGER Backup_Client
AFTER UPDATE OR DELETE ON Client
FOR EACH ROW
BEGIN
    INSERT INTO BackupClient VALUES (:OLD.idClient, :OLD.nom, :OLD.prenom,
SYSDATE, USER);
END;
```

4. Trigger pour générer automatiquement un idClient unique

```
CREATE TRIGGER Generate_Client_ID
BEFORE INSERT ON Client
FOR EACH ROW
BEGIN
    SELECT CONCAT(SUBSTR(:NEW.nom,1,3), SUBSTR(:NEW.prenom,1,3),
TO_CHAR(SQ_CLIENT_ID.NEXTVAL, '000'))
    INTO :NEW.idClient FROM DUAL;
END;
```